



RAJALAKSHMI ENGINEERING COLLEGE

DEPARTMENT OF COMPUTER SCIENCE AND DESIGN

MINI PROJECT

Submitted by

KAMALESH R 231701025

in partial fulfillment for the course

**AI23331 - FUNDAMENTALS OF
MACHINE LEARNING**

Academic Year 2025–2026

BONAFIDE CERTIFICATE

Certified that this project report titled “**User Behaviour Analytics using Machine Learning and Splunk**” is the Bonafide work of **KAMALESH R (231701025)** who carried out the project work for the course **AI23331 - Fundamentals of Machine Learning** under my supervision.

SIGNATURE

Mrs. D. Kalpana

Supervisor

Assistant Professor

Computer Science and Design

Rajalakshmi Engineering College,

Chennai – 602105

Submitted to Project and Viva Voce Examination for the subject AI23331 - Fundamentals of Machine Learning held on _____.

Internal Examiner

External Examiner

ACKNOWLEDGEMENT

Initially we thank the Almighty for being with us through every walk of our life and showering his blessings through the endeavour to put forth this report. Our sincere thanks to our Chairman **Mr.S.Meganathan, B.E, F.I.E.**, our Vice Chairman **Mr. Abhay Shankar Meganathan, B.E., M.S.**, and our respected Chairperson **Dr. (Mrs.) Thangam Meganathan, Ph.D.**, for providing us with the requisite infrastructure and sincere endeavouring in educating us in their premier institution.

Our sincere thanks to **Dr. S. N. Murugesan, M.E., Ph.D.**, our beloved Principal, for his kind support and facilities provided to complete our work in time. We express our sincere thanks to **Prof. Uma Maheswar Rao.**, Associate Professor and Head of the Department of Computer Science and Design for his guidance and encouragement throughout the project work. We convey our sincere and deepest gratitude to our project coordinator and supervisor, **Mrs. D. Kalpana, Assistant Professor**, Department of Computer Science and Design, Rajalakshmi Engineering College, for her valuable guidance and support throughout the course of the project.

KAMALESH R (231701025)

Index

S.No	Section Title	Page No.
1	Introduction	
2	Objectives	
3	Methodology	
3.1	Static Log Analysis	
3.2	Dynamic Log Generation	
3.3	Data Preprocessing	
3.4	Feature Engineering	
3.5	Isolation Forest Model	
3.6	Hybrid Detection	
3.7	Risk Scoring System	
3.8	Real-Time Monitoring	
3.9	Splunk Integration	
3.10	Dashboard and Alerts	
4	Code	
4.1	Import Libraries	
4.2	Live Log Generator	
4.3	Feature Engineering Module	

4.4	Model Training	
4.5	Risk Scoring Logic	
4.6	Dashboard Queries	
4.7	Email Alert Configuration	
5	Results and Output	
5.1	Static Analysis Results	
5.2	Dynamic Analysis Results	
5.3	Dashboard Output	
5.4	Alert Output	
5.5	Key Findings	
6	Conclusion	
6.1	Future Work	
7	References	

MINI PROJECT REPORT

USER BEHAVIOUR ANALYTICS USING MACHINE LEARNING AND SPLUNK

1. INTRODUCTION

User Behaviour Analytics (UBA) is a cybersecurity technique used to monitor and analyze user activities in order to identify suspicious or abnormal behavioural patterns. In modern organizations, cyber threats such as insider attacks, account compromise, unauthorized access, and brute-force login attempts are increasing rapidly.

Traditional security systems mainly depend on predefined rules and signatures. However, these systems are often unable to detect unknown or advanced attacks. Machine Learning provides a powerful solution by identifying unusual behavioural patterns automatically.

This project implements a complete User Behaviour Analytics system using:

- Real-time log generation
- Feature engineering
- Isolation Forest anomaly detection
- Rule-based threat detection
- Risk scoring system
- Splunk dashboard visualization
- Automated email alerting

The project supports both static analysis and dynamic real-time monitoring.

2. OBJECTIVES

The objectives of this project are:

- To detect abnormal user behaviour using Machine Learning
- To simulate real-time user activity logs
- To implement feature engineering for behavioural analysis
- To use Isolation Forest for anomaly detection
- To combine ML-based and rule-based detection
- To implement a risk scoring mechanism
- To integrate the system with Splunk SIEM
- To build real-time dashboards and alerts
- To generate automated email notifications for suspicious users
- To reduce duplicate alerts using throttling and filtering

3. METHODOLOGY

The methodology of this project is structured into the following stages:

3.1 STATIC LOG ANALYSIS

Initially, the project used a static dataset containing user login information.

Dataset fields include:

- user_id
- login_hour
- failed_attempts
- session_time
- new_ip
- device_change

Static analysis helped understand behaviour patterns and train the initial anomaly detection model.

Advantages:

- Easy experimentation
- Fast testing
- Better understanding of features

3.2 DYNAMIC LOG GENERATION

To simulate a real-world environment, a dynamic log generator was developed using Python.

The generator continuously creates login events for:

Normal users:

- Daytime login
- Low failed attempts
- Normal session duration

Suspicious users:

- Night-time login
- High failed attempts

- New IP address
- Device change

Generated logs are continuously stored in: live_logs.csv

This transformed the project into a real-time cybersecurity monitoring system.

3.3 DATA PREPROCESSING

To ensure reliable model performance, preprocessing steps were performed:

- Handling missing values
- Verifying data types
- Cleaning invalid entries
- Scaling numerical features

StandardScaler was used to normalize features so that all features contribute equally to the model.

3.4 FEATURE ENGINEERING

The Feature Engineering converts raw logs into meaningful behavioural features.

The following engineered features were created:

- night_login
- rule_flag

night_login:

- Login before 6 AM is considered suspicious.

rule_flag:

- User is flagged if:
- failed_attempts > 3
- OR night_login = 1
- OR new_ip = 1

3.5 ISOLATION FOREST MODEL

The project uses the Isolation Forest algorithm for anomaly detection.

Reason for choosing Isolation Forest:

- Works without labeled data
- Efficient for anomaly detection
- Fast execution
- Suitable for cybersecurity data
- Detects unusual patterns effectively

Model parameters:

- `n_estimators = 100`
- `contamination = 0.2`
- `random_state = 42`

The model isolates abnormal user behaviour and identifies anomalies.

Output:

- `-1` → Anomaly
- `1` → Normal

3.6 HYBRID DETECTION

The project combines:

- Machine Learning detection
- Rule-based detection

Final anomaly logic:

1. ML detects anomaly
2. Rule-based detection flags user

Then: `final_anomaly = 1`

This hybrid approach improves reliability and reduces false negatives.

3.7 RISK SCORING SYSTEM

A risk scoring system was implemented to prioritize threats.

Risk Levels:

Condition	Score
Failed attempts > 3	+30
New IP address	+20
Device change	+20
Night login	+20
ML anomaly	+40

Score Range	Risk Level
0 – 39	Low
40 – 69	Medium
70+	High

This helps identify the most dangerous users quickly.

3.8 REAL-TIME MONITORING

The project continuously processes logs using three Python modules:

1. Log Generator
2. Feature Engineering Pipeline
3. Anomaly Detection Engine

The pipeline continuously updates the final output file.

The system uses append mode for continuous streaming and Splunk compatibility.

3.9 SPLUNK INTEGRATION

Splunk Enterprise was integrated for monitoring and visualization.

The file: final_output.csv

is continuously monitored by Splunk.

Features used in Splunk:

- Data ingestion
- Indexing
- Search Processing Language (SPL)
- Dashboards
- Alerts
- Email notifications

Index used: index=uba

3.10 DASHBOARD AND ALERTS

A real-time security dashboard was created in Splunk.

Dashboard Panels:

- Total Events
- High Risk Users
- Medium Risk Users
- Low Risk Users
- Risk Distribution Pie Chart
- Risk Trend Line Chart
- Top Suspicious Users Bar Chart
- High Risk User Table

Auto-refresh was enabled using: <refresh>10</refresh>

Email alerts are triggered when:

risk_level = High

Features:

- Dynamic user ID in email
- Cron scheduling
- Deduplication
- Throttling

4. CODE

4.1 IMPORT LIBRARIES

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import time
import os
from datetime import datetime
from sklearn.ensemble import IsolationForest
from sklearn.preprocessing import StandardScaler
```

Libraries used:

- **Pandas → Data processing**
- **NumPy → Numerical operations**
- **Matplotlib → Visualization**
- **Scikit-learn → Machine Learning**

4.2 LIVE LOG GENERATOR

```
while True:
    log = generate_log()
    pd.DataFrame([log]).to_csv(file_name, mode="a",
header=False, index=False)
    time.sleep(2)
```

This continuously generates user activity logs.

4.3 FEATURE ENGINEERING MODULE

The class imbalance between fraudulent and normal transactions is visualized:

```
df["night_login"] = np.where(df["login_hour"] < 6, 10)
```

Rule-based detection:

```
(df["failed_attempts"] > 3)
```

This module prepares the data for Machine Learning.

4.4 MODEL TRAINING

```
model = IsolationForest(
    n_estimators=100,
    contamination=0.2,
    random_state=42
)
```

Feature scaling:

```
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
```

Prediction:

```
preds = model.predict(X_scaled)
```

4.5 RISK SCORING LOGIC

```
if row["failed_attempts"] > 3:
    score += 30
```

Risk level classification:

```
if score >= 70:  
    return "High"
```

4.6 DASHBOARD QUERIES

High Risk Users:

```
index=uba risk_level=High
```

Risk Distribution:

```
index=uba | stats count by risk_level
```

Risk Trend:

```
index=uba | timechart count by risk_level
```

Top Suspicious Users:

```
index=uba risk_level=High  
| stats count by user_id  
| sort -count  
| head 10
```

4.7 EMAIL ALERT CONFIGURATION

Alert Query:

```
index=uba risk_level=High earliest=-1m  
| sort -risk_score  
| head 1  
| table user_id risk_score risk_level login_hour failed_attempts
```

Cron Schedule:

```
*/1 * * * *
```

Dynamic Subject:

 High Risk User Detected: \$result.user_id\$

5. RESULTS AND OUTPUT

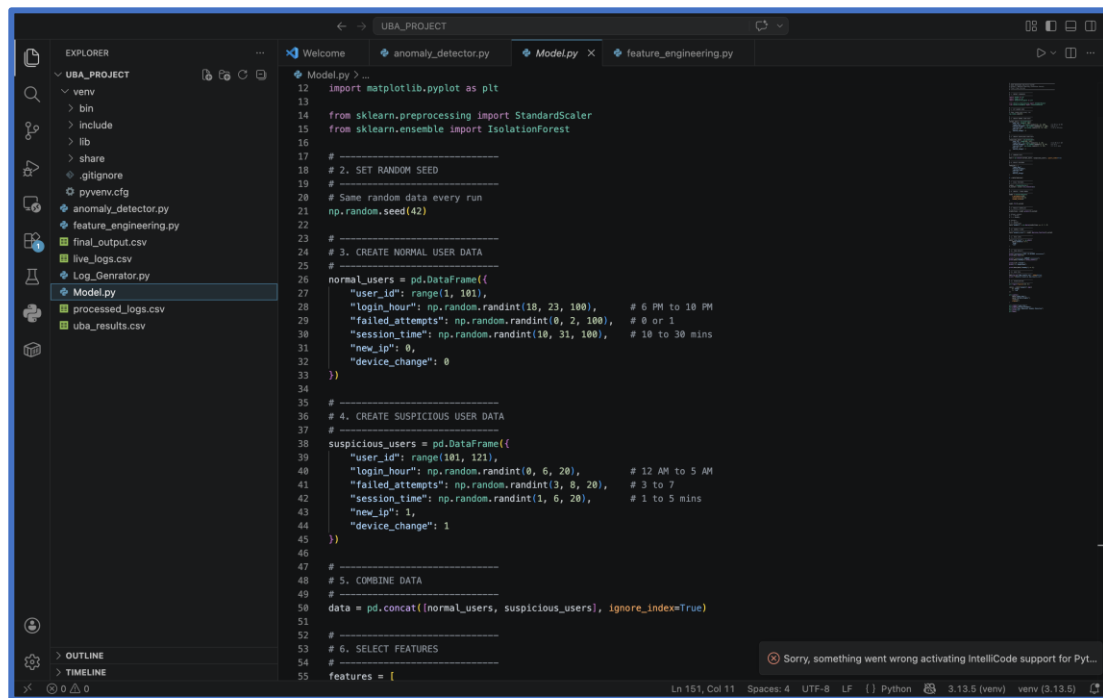
The following results were obtained after implementing the real-time User Behaviour Analytics system.

5.1 STATIC ANALYSIS RESULTS

The static dataset successfully identified suspicious users using Isolation Forest.

Results:

- Normal users classified correctly
- Suspicious users detected
- Anomaly scores generated



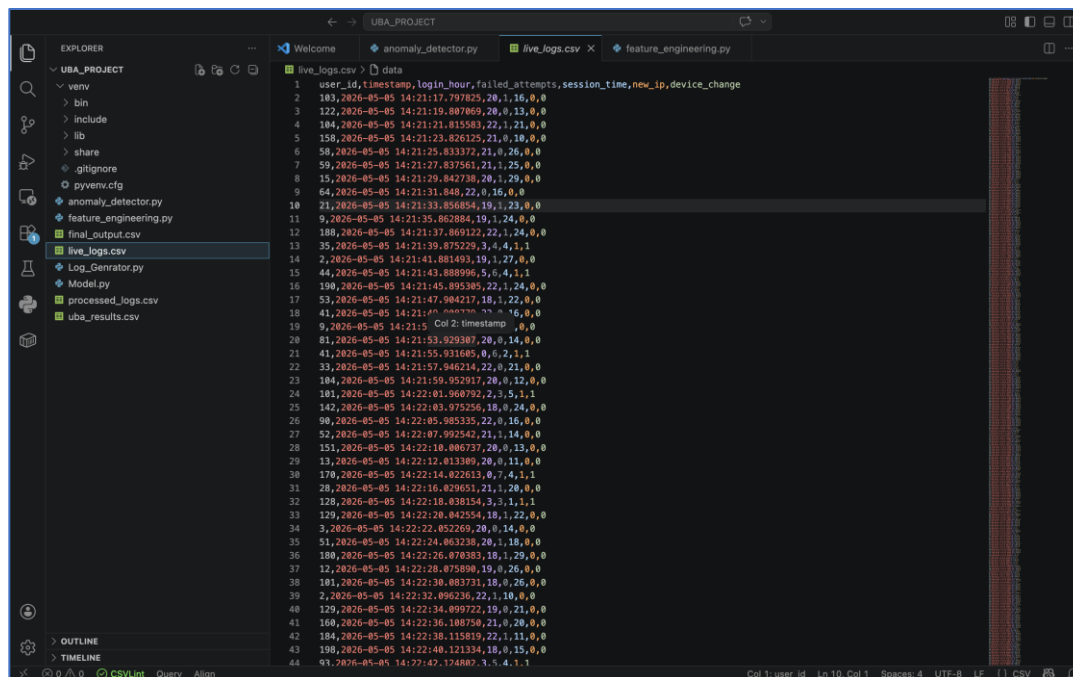
```
12 import matplotlib.pyplot as plt
13
14 from sklearn.preprocessing import StandardScaler
15 from sklearn.ensemble import IsolationForest
16
17 # -----
18 # 2. SET RANDOM SEED
19 # -----
20 # Same random data every run
21 np.random.seed(42)
22
23 # -----
24 # 3. CREATE NORMAL USER DATA
25 # -----
26 normal_users = pd.DataFrame({
27     "user_id": range(1, 101),
28     "login_hour": np.random.randint(18, 23, 100), # 6 PM to 10 PM
29     "failed_attempts": np.random.randint(0, 2, 100), # 0 or 1
30     "session_time": np.random.randint(10, 31, 100), # 10 to 30 mins
31     "new_ip": 0,
32     "device_change": 0
33 })
34
35 # -----
36 # 4. CREATE SUSPICIOUS USER DATA
37 # -----
38 suspicious_users = pd.DataFrame({
39     "user_id": range(101, 121),
40     "login_hour": np.random.randint(0, 6, 20), # 12 AM to 5 AM
41     "failed_attempts": np.random.randint(3, 8, 20), # 3 to 7
42     "session_time": np.random.randint(1, 6, 20), # 1 to 5 mins
43     "new_ip": 1,
44     "device_change": 1
45 })
46
47 # -----
48 # 5. COMBINE DATA
49 # -----
50 data = pd.concat([normal_users, suspicious_users], ignore_index=True)
51
52 # -----
53 # 6. SELECT FEATURES
54 # -----
55 features = [
```

5.2 DYNAMIC ANALYSIS RESULTS

The real-time pipeline continuously generated and analyzed user behaviour logs.

Results:

- Continuous log generation
- Real-time anomaly detection
- Live risk scoring
- Dynamic updates in Splunk



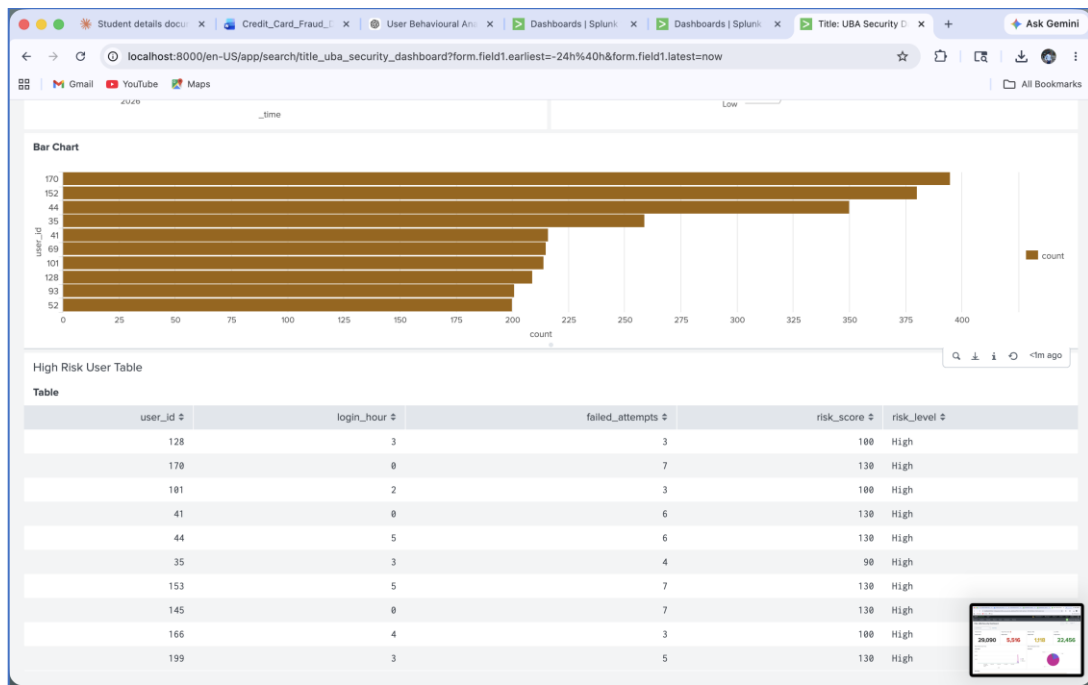
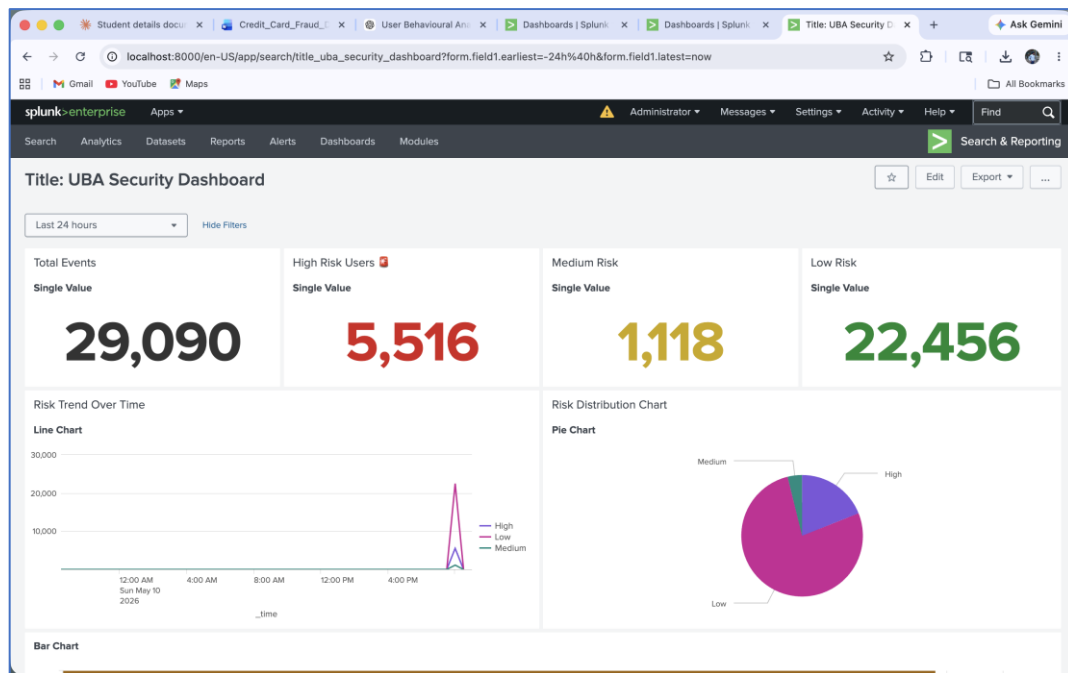
```
1 user_id,timestamp,login_hour,failed_attempts,session_time,new_ip,device_change
2 103,2026-05-05 14:21:17.797825,20,1,16,0,0
3 122,2026-05-05 14:21:19.807069,20,0,13,0,0
4 104,2026-05-05 14:21:21.815583,22,1,21,0,0
5 150,2026-05-05 14:21:23.826325,21,0,10,0,0
6 54,2026-05-05 14:21:25.833372,21,0,26,0,0
7 59,2026-05-05 14:21:27.837561,21,1,25,0,0
8 15,2026-05-05 14:21:29.842738,20,1,29,0,0
9 64,2026-05-05 14:21:31.848,22,0,16,0,0
10 21,2026-05-05 14:21:33.856854,19,1,23,0,0
11 9,2026-05-05 14:21:35.862884,19,1,24,0,0
12 188,2026-05-05 14:21:37.869122,22,1,24,0,0
13 35,2026-05-05 14:21:39.875229,3,4,4,1,1
14 2,2026-05-05 14:21:41.881493,19,1,27,0,0
15 44,2026-05-05 14:21:43.888996,5,6,4,1,1
16 190,2026-05-05 14:21:45.895385,22,1,24,0,0
17 53,2026-05-05 14:21:47.904217,18,1,22,0,0
18 41,2026-05-05 14:21:49.911605,0,6,2,1,1
19 9,2026-05-05 14:21:5,Col 2: timestamp,0,0,0
20 81,2026-05-05 14:21:53.929387,20,0,14,0,0
21 41,2026-05-05 14:21:55.931605,0,6,2,1,1
22 33,2026-05-05 14:21:57.946214,22,0,21,0,0
23 104,2026-05-05 14:21:59.952917,20,0,12,0,0
24 101,2026-05-05 14:22:01.960792,2,0,5,1,1
25 142,2026-05-05 14:22:03.972556,18,0,24,0,0
26 90,2026-05-05 14:22:05.985335,22,0,16,0,0
27 52,2026-05-05 14:22:07.992542,21,1,14,0,0
28 151,2026-05-05 14:22:10.006737,20,0,13,0,0
29 13,2026-05-05 14:22:12.013389,20,0,11,0,0
30 170,2026-05-05 14:22:14.022613,0,7,4,1,1
31 20,2026-05-05 14:22:16.029631,21,1,20,0,0
32 120,2026-05-05 14:22:18.038154,3,1,1,1,1
33 129,2026-05-05 14:22:20.042554,18,1,22,0,0
34 3,2026-05-05 14:22:22.052269,20,0,14,0,0
35 51,2026-05-05 14:22:24.063238,20,1,18,0,0
36 180,2026-05-05 14:22:26.070383,18,1,29,0,0
37 12,2026-05-05 14:22:28.075690,19,0,26,0,0
38 101,2026-05-05 14:22:30.083731,18,0,26,0,0
39 2,2026-05-05 14:22:32.096236,22,1,10,0,0
40 129,2026-05-05 14:22:34.099722,19,0,21,0,0
41 160,2026-05-05 14:22:36.108750,21,0,20,0,0
42 184,2026-05-05 14:22:38.115819,22,1,11,0,0
43 190,2026-05-05 14:22:40.121334,18,0,15,0,0
44 61,2026-05-05 14:22:42.124882,1,5,4,1,1
```

5.3 DASHBOARD OUTPUT

The dashboard successfully displayed:

- Real-time user activity
- Risk distribution
- Threat trends
- High-risk users

Dashboard auto-refresh updated every 10 seconds.



5.4 ALERT OUTPUT

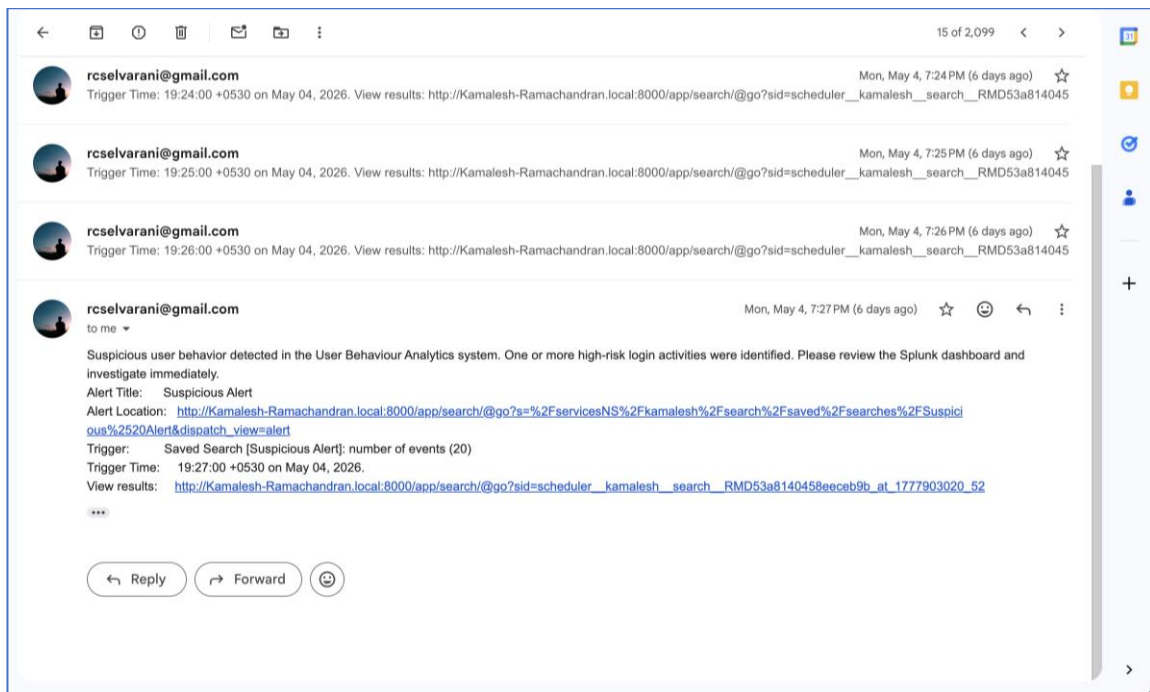
Automated email alerts were successfully generated for high-risk users.

The email included:

- User ID
- Risk Score
- Risk Level
- Login Hour
- Failed Attempts

Duplicate alerts were reduced using:

- earliest=-1m
- dedup user_id
- throttling



6. CONCLUSION

This project successfully developed a complete User Behaviour Analytics system using Machine Learning and Splunk.

The system combines:

- Real-time log generation
- Feature engineering
- Isolation Forest anomaly detection
- Rule-based detection
- Risk scoring
- Splunk monitoring
- Dashboard visualization
- Automated email alerts

The project demonstrates how Machine Learning can be integrated with SIEM systems to detect suspicious user behaviour in real time.

6.1. Future Work

Future improvements include:

- Deep Learning models (LSTM, Autoencoders)
- Cloud deployment using AWS/Azure
- Real-time API integration
- Geo-location based detection
- User behaviour profiling
- Large-scale enterprise deployment

7. REFERENCES

- Scikit-learn Documentation
- Splunk Enterprise Documentation
- Python Documentation
- Pandas Documentation
- NumPy Documentation
- Isolation Forest Research Paper
- Machine Learning for Anomaly Detection
- Cybersecurity SIEM Concepts
- Splunk SPL Documentation